

Föreläsning 16

Ändrad 1 Oct 14:07

Nu följer ett program som skriver ut inmatad text baklänges. Så här blir det när man kör:

```
1 Hej, jag heter Janne!
2 ADOLF I PARIS RÖR SIRAP I FLODA
3 Hejdå!
4 ^d
5 !ådjeH
6 ADOLF I PARIS RÖR SIRAP I FLODA
7 !ennaJ reteh gaj ,jeH
```

Själva programmet använder indexerade `char`-variabler:

```
1 #include <stdio.h>
2
3 void main()
4 {
5     char ch, s[2000];
6     int i = 0, j;
7
8     while ((ch = getchar()) != EOF)
9         s[i++] = ch;
10
11     for (j = i - 1; j >= 0; j--)
12         putchar(s[j]);
13
14     putchar('\n');
15 }
```

- Ni ser `i++` inuti indexet. Sånt ser man ofta i C-program. C-programmerare skriver inte gärna en rad i onödan.

Repetition: `i++` är ett uttryck vars värde är lika med `i` *före* ökningen.

- Inläsningsloopen läser in hela användarens inmatning. Alla tecken hamnar i `s`, med radslut och allt.

- Ni hade inte sett loopar som vandrade baklänges förut. Nu har ni sett det: `j--` i stället för `j++`.

- Denna gång kan inte datorn skriva ut någonting innan användaren tryckt `[Control]+[d]`. Detta eftersom utskriftsloopen ligger efter inläsningsloopen.

- Övning: Skriv ett program som skriver ut användarens inmatning, två gånger.

- Övning: Den ännu latare C-programmeraren skriver `while`-loopen på en enda rad -- utan loopkropp, och utan `ch`. Hur?

Sortering igen

Nu är ni mogna för det riktiga sorteringsprogrammet. Här är output, från osorterat till sorterat:

```

1 36 74 52 34 90 54 57 43 84 78 66 64 90 67 32
2 32 34 36 43 52 54 57 64 66 67 74 78 84 90 90

```

Programmet:

```

1  #include <stdio.h>
2
3  void print(int n, float x[])
4  {
5      int i;
6      for (i = 0; i < n; i++)
7          printf("%g ", x[i]);
8      printf("\n");
9  }
10
11
12 void swap(float *x, float *y)
13 {
14     int temp;
15
16     temp = *x;
17     *x = *y;
18     *y = temp;
19 }
20
21
22 void order(float *x, float *y)
23 {
24     if (*x > *y)
25         swap(x, y); /* &*(x) = x */
26 }
27
28
29 void sort(int n, float x[])
30 {
31     int i, j;
32
33     for (i = 0; i < n - 1; i++)
34         for (j = i + 1; j < n; j++)
35             order(&x[i], &x[j]);
36 }
37
38
39 void main()
40 {
41     int n = 15;
42     float x[] = {36, 74, 52, 34, 90,
43                 54, 57, 43, 84, 78,
44                 66, 64, 90, 67, 32};
45
46     print(n, x);
47     sort(n, x);
48     print(n, x);
49 }

```

■ Funktionerna `swap()` och `order()` är desamma som tidigare.

■ I huvudfunktionen `main()` deklareras femton `float`-variabler. De heter `x[0]`, `x[1]`, `x[2]`, osv till `x[14]`.

En ny detalj är att jag aldrig ber om femton variabler i deklarationen. Det är initieringen som gör det! Kompilatorn reserverar plats för de värden jag räknar upp.

■ Jag deklarerar och initierar `n` till 15 för att hålla reda på antalet element i `x` -- eller *längden* av `x` som man ofta säger.

■ Funktionen `print()` behöver `n` som parameter för att veta hur många element den ska skriva ut.

Programmeraren måste se till att inte `x` får index utöver

dem som bokats. Inget hindrar `print()` från att skriva ut `x[1000]`.

En kompilator som varnar för såna misstag är felaktig! C-programmeraren *ska* kunna använda pekare och index utan inskränkningar.

■ `sort()` innehåller inget nytt. Exempelvis såg ni i ett tidigare exempel hur lämpliga index för sorteringen kan genereras. Men den här gången skriver vi inte ut indexen -- vi använder dem som index till `x`.

index och pekare

Här följer ytterligare en demonstration av likheten mellan pekare och indexerade variabler.

Funktionen `print()` skulle kunna implementeras på följande sätt:

```

3 void print(int n, float x[])
4 {
5     float *last = &x[n];
6
7     do
8         printf("%g ", *x);
9     while (++x != last);
10
11     printf("\n");
12 }
```

Men gör nu inte på ovanstående sätt, bara för att det är möjligt! Det är inte särskilt lättläst!

Någon kanske tror att hans program blir snabbare i den varianten. Det kanske det blir också, eftersom `x[i]` betyder `*(x + i)` i den förra versionen. Datorn slipper denna addition i den nya versionen.

Å andra sidan kan man aldrig lita på att en kompilator är särskilt smart. Den gamla versionen kan vara snabbare!

Men hursomhelst, att försöka optimera C-kod genom såna omskrivningar är nästan aldrig meningsfullt. Vinsten är inte värd den försämrade läsbarheten.

Om maximalt snabb kod är absolut nödvändig i något sammanhang måste programmeraren övergå till assembler. Programspråket C duger inte.

■ Övning: Ovanstående optimeringsförsök är definitivt inte meningsfullt. Varför?

Samma lektion

